Formalizing Delcourt–Postle's refined absorption in Lean: formalization companion

Juan Pablo Traverso Gianini
Independent researcher, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

Companion to *Formalizing Delcourt–Postle's refined absorption in Lean*, research note, preprint 1.0.1.

Preprint 1.0.1, 27 October 2026.

Version 1.0.1 DOI: 10.5281/zenodo.23715622.

© 2026 Juan Pablo Traverso Gianini. Licensed under CC BY 4.0.

Abstract

This companion locates and explains how to reproduce the Lean proofs accompanying the note on Delcourt–Postle and Postle's survey. It relates the cited results to their formal declarations, specifies their hypotheses, and records the external proofs reused. It distinguishes the original formal archive, containing 316 sources, from the later unified comparison, which adds one source and selects eighteen declarations. The full preflight for that comparison passed; its report is preserved separately from the Palomar review and registration process. Appendix A explains the omni-absorber contract and its degree bound. This revision changes only the documents: the Lean sources and their evidence archives are preserved.

1. Mathematical sources and scope

The two principal texts are Delcourt–Postle [1], with statement baseline arXiv:2402.17855v1, and Postle [2], developed against arXiv:2510.19978v1. The survey's published identifier is DOI 10.1137/25M1805977, ICM 2026, Volume 6, pages 212–231. Its Conjecture 1.1, Theorem 1.10, Definitions 3.6–3.7 and Lemma 3.8 were checked in that edition. This is a check of the identified statements, not whole-document equivalence between editions.

The formalization contains the principal absorption, dense decomposition and design-existence endpoints listed below. The arbitrary-index endpoint proves existence of simple block families for all fixed $q > r \geq 2$ and $\lambda \geq 1$ at all sufficiently large admissible orders. It is not restricted to the Steiner case. General high-girth applications remain outside this frozen scope. This is an intermediate documentary revision; the planned definitive version will describe full coverage of both source texts after the remaining proofs and their dependencies are completed and audited.

The source theorem authors retain mathematical attribution. The note explains interfaces and derives consequences from proved inputs. Classical triple-system intersection work [10,11] is acknowledged; historical priority for each general-parameter consequence is not asserted. The Nash–Williams input is attributed to Delcourt–Postle [7], with the reused formal proof documented in [8,9].

2. Canonical endpoints

The tables identify the eighteen declarations selected by the unified comparison. Their full types are in `Palomar/UnifiedChallenge.lean`; the earlier extraction spans and hashes remain in `provenance/palomar-extractions.json`. Each selected name occurs in the frozen axiom-query outputs. The library also proves results outside this selection.

2.1. Original article

Source result	Canonical declaration
[1, Theorem 1.7]	`RefinedAbsorption.Faithful2024.theorem_1_7`
[1, Theorem 1.11]	`RefinedAbsorption.Faithful2024.theorem_1_11`
[1, Theorem 6.1]	`RefinedAbsorption.Original2024.theorem_6_1`
[1, Theorem 2.8]	`RefinedAbsorption.Pending.theorem_2_8`
[1, Theorem 6.2], ambient clause exposed	`RefinedAbsorption.Faithful2024.theorem_6_2_ambient`
[1, Theorem 6.3]	`RefinedAbsorption.Pending.theorem_6_3`

The first two endpoints retain explicit host vertex sets and the distinction between bounded edge usage, bounded member size and the refinement degree of a family. The reserve theorem uses labelled edge copies, degree bounds on the designated root set and the stated ambient intersection. It does not substitute global near-regularity or an edge-count cap for that contract.

The old `RefinedAbsorption.Pending.theorem_6_2` remains available. Its absorber predicate uses the edge support of the old graph. The additive endpoint also forbids an absorber edge wholly contained in the full old vertex set. It includes isolated labels and permits mixed old/new edges. The inherited namespace `Pending` is not a statement that these proved endpoints remain open.

2.2. Survey

Source or qualification	Canonical declaration
[2, Theorem 1.10]	`RefinedAbsorption.Final.theorem_1_10`
[2, Conjecture 1.1], fixed index at least one	`RefinedAbsorption.existenceConjecture_proved`
Steiner existence specialization	`RefinedAbsorption.Final.existenceConjecture_steiner`
Negation of literal global embedding conclusion	`RefinedAbsorption.FaithfulSurvey2025.not_generalEmbeddingLemmaLiteral`
Proved piecewise replacement	`RefinedAbsorption.FaithfulSurvey2025.generalEmbedding_piecewise`

The last two rows are distinct claims. The global predicate uses old vertices occurring in the indexed pieces. The perfect-matching witness covers all old vertices, so it also refutes the printed conclusion on the full old vertex set. No general definitional identity across every unused-vertex convention is inferred from that witness.

The survey's proof reference [4, Appendix B] was followed. Its proof exists; it also writes a global injection. The piecewise theorem permits separate injective maps, with the host conditions and distinct images of new edges. It must not be reported as a proof of the global assertion.

2.3. Research note

Location in the note	Canonical declaration
Theorem 5.1	`RefinedAbsorption.Extensions.steiner_completion_avoiding`
Corollary 5.2(i)	`RefinedAbsorption.Extensions.steiner_intersection_prescribed`
Corollary 5.2(ii)	`RefinedAbsorption.Extensions.partialSteiner_intersection_embedding`
Proposition 5.3	`RefinedAbsorption.Extensions.steiner_intersection_spectrum_poly`
Theorem 6.2, every existing system	`RefinedAbsorption.Assembly.steiner_quantitatively_almost_resolvable`
Theorem 6.2, existence	`RefinedAbsorption.Assembly.exists_quantitatively_almost_resolvable_steiner`
Theorem 6.3	`RefinedAbsorption.Assembly.nash_williams_scheduled_decomposition`

The intersection endpoints are in `RefinedAbsorption/Extensions/Intersection.lean`. They use the compatible decomposition theorem, ordinary sparse completion and finite packing lemmas. The three closed quantitative endpoints are in `RefinedAbsorption/Assembly.lean`. Their older conditional predecessors are retained unchanged.

Theorem 6.1 of the note is the input `Pseudorandom.chromatic_index_polynomial`, from the pinned source described in Section 4; it is not an additional declaration in the eighteen-statement selection.

3. Quantifiers and representation

For completion and prescribed sparse intersections, fix $q > r \geq 1$ first. The positive sparsity constant and order threshold are chosen before the ambient order and the partial systems. Sparsity concerns $\Delta_{r-1}(H(P))$, the covered r-graph. The forbidden partial system need not be sparse. The additional crossing guarantee is restricted to forbidden blocks whose r-edges avoid $H(P)$.

For arbitrary-index existence, fix $q > r \geq 2$ and $\lambda \geq 1$ before choosing the order threshold. The threshold may depend on all three parameters. The admissibility condition uses that index; no uniform threshold over all indices is asserted.

For finite exact-intersection embedding, a given finite partial system determines an order threshold. Every larger admissible order admits an injection and two systems with precisely the image block family in common. The injection is chosen with the systems; the endpoint does not quantify over every previously prescribed injection. The spectrum endpoint chooses both systems and gives an initial interval $0 \leq k \leq cn^r$.

The universal Steiner-colouring result fixes $q > r \geq 2$, then chooses γ, n_0 before n and the Steiner system. The existence variant adds admissibility. Lean writes the point-degree quotient as natural division before casting to the reals:

$$R = \frac{\binom{n-1}{r-1}}{\binom{q-1}{r-1}}.$$

The counting proof establishes exact divisibility for a Steiner system, so this is the ordinary point degree, rather than a floor approximation. The formal colour map is defined on finite

vertex sets and is constrained on the system's blocks. Its number of labels satisfies

$$R \leq m \leq R + R^{1-\gamma}.$$

Unused labels are allowed. The note consequently states an upper bound on the number of partial parallel classes, with the lower bound justified separately.

For the graph application there are universal constants γ, n_0 before the finite graph. The hypotheses are triangle divisibility, sufficiently large order and the exact inequality

$$3n \leq 4\delta(G).$$

The output is an actual triangle decomposition and a proper colouring of its triangle hypergraph, with

$$\Delta(G)/2 \leq m \leq \Delta(G)/2 + (\Delta(G)/2)^{1-\gamma}.$$

No near-regularity hypothesis on the graph, external theorem premise or spanning-factor conclusion appears. Both quantitative applications have $0 < \gamma < 1$.

Finite types and decidable-equality instances are representation parameters. The selected contracts use ordinary finite types in universe zero. Block families are finite sets, so the fixed-index existence result does not allow repeated blocks. The name `steiner_orthogonal_mate` elsewhere in the library denotes bounded cross-intersections and no common blocks; classical Steiner orthogonality is not asserted.

4. Proved external assembly

The source pin for Pippenger–Spencer is

`b19273396d369332fb489fe07e61e27c9e7f9233`

in `jtraverso/pippenger-spencer`. The installed cut contains `Basic.lean`, `Colour Hypergraph.lean` and `Polynomial.lean` under `RefinedAbsorption/Upstream/Pippenger Spencer/`. Only import routes and line endings changed. Nineteen shared matching sources were verified identical after import-prefix substitution and are reused from the Nash–Williams dependency. Original and installed byte hashes, including CRLF/LF normalization, are recorded in `provenance/assembly-sources.json`.

The polynomial colouring endpoint is `Pseudorandom.chromatic_index_polynomial`. It is proved through the Ehard–Glock–Joos input [6], using an auxiliary colour hypergraph and completion. Its exact contract includes maximum degree, polynomial codegrees and an exponential edge-count cap. It has no lower-degree condition. This is not an assertion that [5] prints that exact quantitative formulation.

The Nash–Williams dependency is `jtraverso/nash-williams-lean` at

`2603197abdbf0a600ec663f6f046f70fbd209d4a`.

The actual endpoint is `NashWilliams18.PaperI.nash_williams_via_paperII`, imported from `RequestProject.NW18.PaperIIAssembly`. The local module `RefinedAbsorption.External Proofs` supplies both contracts by direct application of these proved endpoints. The three final declarations in `RefinedAbsorption.Assembly` therefore have neither external theorem hypothesis.

The local module root was relocated from `RequestProject` to `RefinedAbsorption` to avoid a collision with the dependency's module routes. Mathematical declaration namespaces were preserved. The source-cut licence and correspondence records are included in the repository.

5. Build, axioms and preservation

The formal source commit is

`5428ee9b772e67dd7ec6da443199b36a003b72d4`.

It is identified by the local formal-source tag `formal-v1.0`. The toolchain is `leanprover/lean4:v4.35.0-rc3`; Mathlib is pinned to

`c55e6e786f49471c72fbddbec5415808896aec1e`.

The manifest fixes ten Git dependencies, including Nash–Williams. Their actual checkout revisions and clean tracked trees were checked. The default Lake targets are `RefinedAbsorption`, `checks` and `Palomar`. The aggregate root transitively imports all 299 local library modules; together with the root these are 300 sources. The frozen build adds ten check modules and six standalone comparison files, for 316 sources. The later unified comparison brings the current total to 317.

The campaign began without the project build directory and reused caches of pinned dependencies. Bounded builds were resumed until the library and check targets completed; the six comparison interfaces and a successful default build followed. This records a clean project build, not a cold rebuild of every dependency. The failed attempts and their corrections, including module-comment placement and UTF-8 log decoding on Windows, remain in `publication/assembly/`. They are part of the build history, not additional proof assumptions.

After the inputs were committed, the freeze validator compared source and configuration bytes with that commit, checked Lake freshness and genuine traces, and recorded source and artifact hashes. The following table gives the recorded axiom queries.

Check module	Queries
`AxiomCheck`	290
`ExtensionsAxioms`	14
`FaithfulAxioms`	13
`IntersectionAxioms`	11
`Original2024Axioms`	14
`PendingAxioms`	2
`PendingEndpoints`	3
`QuantitativeAxioms`	7
`AssemblyAxioms`	5
`SourceFidelityAxioms`	1
Total	360

These are queries, not a claim of 360 distinct theorem names. The outputs contain only `propext`, `Classical.choice` and `Quot.sound`, or no axioms. The retained proof development has no unresolved proof hole or project axiom. The four `Palomar/*Challenge.lean` files contain 36 `sorry` proof placeholders by design: six, seven, five and eighteen in the Article2024, ResearchNote, Survey2025 and Unified files, respectively. Including one mention in a comment in each file gives 40 textual occurrences. Library, check and Solution modules contain none. The Solution supplies the proofs and does not import the Challenge. “Axiom-free” would be inaccurate.

All 300 original returned Lean sources are retained. The migration’s comparison reports no textual declaration-header differences and no change to 821 comment/whitespace-normalized non-proof declaration spans. The later import relocation preserves all 301 migrated sources modulo its documented prefix substitution. These are textual checks, not a cross-version kernel equivalence theorem. The older direct-Lean migration record, with 301 sources and 354 queries, remains historical evidence and is not substituted for the final Lake certificate.

6. Archive and reproduction

The immutable formal archive is

`publication/formal-freeze/formal-v1.0-5428ee9b772e.tar.gz`,

with SHA-256

`e9bb21b17a110a392a7751afefb9a2e9d6bf218c7e6dac788006234f3af09c05`.

It contains the source commit and the completed build evidence. The external `FREEZE.json` receipt and SHA sidecar identify that archive. The validation record `publication/assembly/VALIDATION.json` has SHA-256

`45321434a170482d8473182ecc8c1fd60b66a84bb9480ef7f6c26f9f2b130157`.

From the complete publication checkout, `python tools/verify_formal_freeze.py` checks current and archived source correspondence, archive integrity, the original-source archive and the recorded axiom logs. It does not execute Lean or independently replay proofs. The formal archive cannot contain its own archive-hash receipt; use the complete publication checkout for that convenience checker.

For a fresh proof reconstruction, use a separate checkout of the formal source commit, or extract the formal archive, install the pinned Lean toolchain, and run:

```
lake exe cache get
lake build
```

The public manifest uses immutable Git revisions. Machine-local override paths used in the recorded Windows campaign are ignored by Git and are not needed by this portable configuration. Compiled artifact hashes describe the recorded platform; a reconstruction on another platform need not reproduce those binary bytes.

The manuscripts are maintained in English and Spanish Markdown, then converted through the author's Nash–Williams series template to final TeX and PDFs. Rendered-page inspection, bilingual parity and document hashes are separate from the formal-source freeze. Later document and metadata commits can retain the frozen Lean/configuration bytes; exact publication and registry commits are recorded in their own receipts.

7. Comparison, review and reuse

Palomar [21] is a registry of formalized mathematical results. A Challenge is a file of statements to be compared; a Solution supplies their proofs. The comparison interface specifies those files and the selected declarations. A preflight is a preliminary verification run. NanoDa [22] and con-ron [23] are additional kernel implementations used to check exported declarations. Their exact binary hashes and the pipeline revision used here are recorded in the archived mechanical report.

The unified entry uses `Palomar/Unified.comparator.json`, metadata `Palomar/Unified/formalization.yaml`, Challenge `Palomar.UnifiedChallenge`, and the `RefinedAbsorption` library as its Solution. It collects the eighteen declarations of Section 2. The configurations `Palomar/Article2024.comparator.json`, `Palomar/Survey2025.comparator.json`, and `Palomar/ResearchNote.comparator.json` remain as historical views of the same selection.

The Challenge imports only Mathlib. Its definitions have complete bodies, and its eighteen theorems have intentional placeholders stating the claims to be compared. The Solution imports the proofs and does not import the Challenge. These placeholders are not used as proof premises.

The unified comparison commit is

`867d1e3d5e48ef20a8f1ae68237834eb06211cbc`.

Its official full preflight, run `40478830948`, completed successfully with no errors on 27 October 2026 at 10:18 UTC. The report records eighteen selected theorems, 317 checked sources, and acceptance by NanoDa and con-ron. The warning about the preferred Challenge size is preserved in the report; the mandatory limit is not exceeded. The records are in `publication/palomar/OFFICIAL_PREFLIGHT.json` and the mechanical report identified there.

The subsequent submission has identifier `jne18fh5pbug`. The inspected local receipt, dated 27 October 2026 at 11:59 UTC, records status `verifying`. This is a historical observation, not a live query. A successful preflight and a submission receipt do not by themselves establish editorial review or final registration. This document revision changes neither the submission nor its receipts.

The author's contributions to Lean Pool [17] provide another precedent for this way of working. The finite nibble and concentration-inequality contributions were merged into the upstream repository. They are cited as related developments, not dependencies imported by this library. The code actually reused here is identified in Section 4: its provenance and revisions form part of the reproducible evidence.

The internal document audit was performed by Codex under the author's direction. It is distinct from the requested external reviews and from independent human peer review. Mathematical and editorial responsibility rests with the author.

Appendix A. The available omni-absorber contract

The declaration `RefinedAbsorption.Faithful2024.theorem_1_7` already allows the omni-absorber to be used independently of its proof. For fixed $q > r \geq 1$ it supplies $C \geq 1$ and $0 < \varepsilon < 1$. If G is an r-graph on n vertices and X is a spanning subgraph of G, the degree hypotheses are

$$\delta_{r-1}(G) \geq (1-\varepsilon)n, \qquad \Delta_{r-1}(X) \leq n/C. \tag{A.1}$$

The output includes an omni-absorber $A \subseteq G$, edge-disjoint from X, on the same vertex set, with

$$\Delta_{r-1}(A) \leq C \max\{\Delta_{r-1}(X),\, n^{1-1/r}\log n\}. \tag{A.2}$$

The decomposition family consists of cliques of size q in $X \cup A$, each containing at most one edge of X. Each edge of $X \cup A$ occurs in at most C members of that family. For each divisible leftover $L \subseteq X$, the decomposition function selects from it a decomposition of $A \cup L$. Divisibility is required of the leftover being absorbed; it is not added as a construction hypothesis on all of X.

Bound (A.2) is the one formalized. Its additional term depends on the order of the host and cannot be omitted when citing the result. If $\Delta_{r-1}(X) \geq n^{1-1/r}\log n$, the maximum reduces to the first term. The theorem does not assert that additional inequality for every sparse subgraph. Flatness belongs to the multi-refiner stage: `RefinedAbsorption.Faithful2024.Theorem_3_8_conclusion` expresses it through `RefinedAbsorption.Original2024.IsFlatTo`. It should not be added as a clause of the omni-absorber output. Likewise, multiplicities of labelled copies and the number of family members using an edge are distinct data.

Acknowledgements

The author is deeply grateful to his wife María Paz and to his children Lucas, Juan Cristóbal, Francisca, Raimundo and Benjamín for their love, patience and support.

Use of artificial intelligence and computational tools

Claude, by Anthropic, and ChatGPT/Codex, by OpenAI, were used to explore and check arguments and to prepare the manuscript. Aristotle, by Harmonic, contributed to exploring approaches, searching for counterexamples, developing proofs, and formalizing and reviewing them in Lean. Its role was not limited to translating finished proofs.

Certo [14] and Jacobian [15] were used to explore finite instances, check identities and assess candidate constructions. These computations informed the development of the arguments; finite checks were not treated as proofs of universal statements.

The author remains responsible for the arguments, citations, code and presentation. No AI system is listed as an author.

References

[1] M. Delcourt and L. Postle, *Refined Absorption: A New Proof of the Existence Conjecture*, arXiv:2402.17855v1 (2024). https://doi.org/10.48550/arXiv.2402.17855.
[2] L. Postle, *Refined Absorption: A New Proof of the Existence Conjecture and Its Applications to Extremal and Probabilistic Design Theory*, Proceedings of the International Congress of Mathematicians 2026, Volume 6, 212–231. https://doi.org/10.1137/25M1805977. Statement baseline: arXiv:2510.19978v1 (2025), https://doi.org/10.48550/arXiv.2510.19978.
[3] M. Delcourt and L. Postle, *Finding an Almost Perfect Matching in a Hypergraph Avoiding Forbidden Submatchings*, arXiv:2204.08981 (2022). https://doi.org/10.48550/arXiv.2204.08981.
[4] M. Delcourt, C. Henderson, T. Lesgourgues and L. Postle, *Erdős meets Nash-Williams*, arXiv:2507.23624v1 (2025), Appendix B. https://doi.org/10.48550/arXiv.2507.23624.
[5] N. Pippenger and J. Spencer, *Asymptotic behavior of the chromatic index for hypergraphs*, Journal of Combinatorial Theory, Series A 51 (1989), 24–42. https://doi.org/10.1016/0097-3165(89)90074-5.
[6] S. Ehard, S. Glock and F. Joos, *Pseudorandom hypergraph matchings*, Combinatorics, Probability and Computing 29(6) (2020), 868–885. https://doi.org/10.1017/S0963548320000280.
[7] M. Delcourt and L. Postle, *A Proof of Nash-Williams' Conjecture*, arXiv:2606.11178v1 (2026). https://doi.org/10.48550/arXiv.2606.11178.
[8] J. P. Traverso Gianini, *Triangle decompositions I: Formalizing the Nash–Williams theorem in Lean*, preprint 1.0 (2026). https://doi.org/10.5281/zenodo.23428173.
[9] J. P. Traverso Gianini, *Triangle decompositions II: Robust absorption for typed triangles*, preprint 1.0 (2026). https://doi.org/10.5281/zenodo.23502224.
[10] C. C. Lindner and A. Rosa, *Steiner triple systems having a prescribed number of triples in common*, Canadian Journal of Mathematics 27(5) (1975), 1166–1175. https://doi.org/10.4153/CJM-1975-122-4.
[11] C. C. Lindner and A. Rosa, *Steiner triple systems having a prescribed number of triples in common (corrigendum)*, Canadian Journal of Mathematics 30(4) (1978), 896. https://doi.org/10.4153/CJM-1978-077-9.
[12] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, Automated Deduction — CADE 28 (2021), 625–635. https://doi.org/10.1007/978-3-030-79876-5_37.
[13] The mathlib Community, *The Lean Mathematical Library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (2020), 367–381. https://doi.org/10.1145/3372885.3373824.
[14] J. P. Traverso Gianini, *Certo: computational tools with verifiable certificates*. Software repository (2026), https://github.com/jtraverso/certo-math. Accessed 27 October 2026; the exploratory version was not recorded.
[15] Morluto and contributors, *Jacobian: composable mathematical tools*. Software repository, https://github.com/morluto/jacobian. Accessed 27 October 2026; the exploratory version was not recorded.

[16] J. P. Traverso Gianini, *The Pippenger–Spencer theorem in Lean 4*. Software repository, https://github.com/jtraverso/pippenger-spencer. Commit `b19273396d369332fb489fe07e61e27c9e7f9233` (2026).
[17] J. P. Traverso Gianini, *Contributions to Lean Pool* (2026). https://github.com/jtraverso/lean-pool. Related contributions: https://github.com/LeanPool/lean-pool/pull/471, https://github.com/LeanPool/lean-pool/pull/431. Both merged; checked 27 October 2026.
[18] P. Keevash, *The existence of designs*, arXiv:1401.3665 (2014), version 4 (2024). https://doi.org/10.48550/arXiv.1401.3665.
[19] S. Glock, D. Kühn, A. Lo and D. Osthus, *The Existence of Designs via Iterative Absorption: Hypergraph F-Designs for Arbitrary F*, Memoirs of the American Mathematical Society 284, no. 1406 (2023). https://doi.org/10.1090/memo/1406.
[20] M. Molloy and B. Reed, *Near-optimal list colorings*, Random Structures & Algorithms 17(3–4) (2000), 376–402. https://doi.org/10.1002/1098-2418(200010/12)17:3/4%3C376::AID-RSA10%3E3.0.CO;2-0.
[21] Palomar Registry, *About Palomar* (2026). https://palomar-registry.org/about.
[22] NanoDa contributors, *nanoda_lib: external type checker for Lean 4*. https://github.com/ammkrn/nanoda_lib.
[23] Lean contributors, *con-ron*. https://github.com/leanprover/con-ron.